SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	S.Venkateswarlu	**Reg. No.:**	192425340
Date:	28-10-26	**Duration:**	60 minutes

Code of Conduct

I ___S.Venkateswarlu (192425340)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___ *S.Venkateswarlu* _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improve Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement ana incomplete, inac lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable	Architecture is partially suitable with limited justification.	Inappropriate archi selected or justifica missing.

		justification.		
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is inco unclear, or techn incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component inte and quality attrib poorly explained missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is w absent; report la organization and technical clarity

Criteria	**Mark**

System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Industry Context: International seaports are highly complex bottlenecks where giant container ships must be loaded and unloaded quickly. Inefficient container yard management leads to crane delays, traffic jams for terminal trucks, and long ship wait times, costing shipping companies thousands of dollars per hour. Port operators need a centralized, automated orchestration system that schedules crane movements and container placements to maximize port speed.

.

Problem Statement: Design and develop a highly responsive Seaport Container Yard Coordination Platform that orchestrates container movements, crane schedules, and cargo truck traffic. The system must process real-time inputs—including ship cargo configurations, automated guided vehicle (AGV) paths, container yard storage grids, and import/export timelines. It must compute optimal crane picking patterns to minimize ship unloading times.

.

Objectives:

o
Balance
container placements inside yard storage grids to minimize redundant movements.

o
Coordinate
crane movements in real time to avoid collisions and mechanical downtime.

o
Route
automated terminal trucks through cargo yards using dynamic traffic paths.

o
Adapt
yard logistics automatically when ship arrivals shift due to weather delays.

o
Provide
live operational dashboards for port logistics and customs management teams.

.

Expected Outcomes:

o
Shorter
turnaround times for cargo ships at international maritime shipping ports.

o
Maximum
performance and utilization from high-value port cranes and automated trucks.

o
Lower
traffic gridlock and fuel waste for terminal transport vehicles inside ports.

o
Increased
cargo processing capacities for ports without expanding physical footprints.

Architecture Design Report

Seaport Container Yard Coordination Platform

Selected Architecture: Event-Driven Microservices Architecture (Hybrid with Layered Structure)

1. Analyze System Requirements

1.1 System Objectives

Port terminals are bottlenecks where large container ships must be loaded and unloaded quickly. Poor yard management causes crane delays, truck traffic jams and long ship waiting times. The platform must orchestrate container movements, crane schedules and truck traffic in real time. Its objectives are to:

- Balance container placements in yard storage grids to minimize redundant movements.
- Coordinate crane movements in real time to avoid collisions and mechanical downtime.
- Route automated terminal trucks and AGVs through the yard using dynamic traffic paths.
- Adapt yard logistics automatically when ship arrivals shift because of weather delays.
- Provide live dashboards for port logistics and customs management teams.
- Compute crane picking patterns that minimize ship unloading time.

1.2 Functional Requirements Influencing the Architecture

Functional Requirement	Architectural Impact
Ingest ship cargo plans, yard grid state, AGV paths and import/export timelines	Needs an integration layer with adapters and a streaming pipeline for continuous inputs.
Yard slot allocation and re-handling minimization	Independent optimization service with access to live yard state.
Crane scheduling and collision avoidance	Low-latency service reacting to crane position events.
Dynamic AGV / truck routing	Routing service consuming vehicle position streams and congestion data.
Re-plan when vessel arrival shifts	Event-triggered workflow that notifies several services at once.
Live dashboards for logistics and customs	Push-based presentation layer (WebSocket) with role-based views.
Customs hold / release tracking	Separate service and event stream so held containers affect all plans.

Audit and administration	Append-only log store, configuration service and access control.

1.3 Non-Functional Requirements and Key Quality Attributes

Quality Attribute	Requirement from the Scenario	Priority
Performance	Real-time re-planning and dashboard updates for fast-changing yard events.	Very high
Scalability	Many cranes, hundreds of AGVs and tens of thousands of containers per port; multiple berths.	High
Reliability	Cranes and trucks cannot stop when one component fails; safe fallback needed.	Very high
Availability	Ports operate 24x7; downtime costs thousands of dollars per hour.	Very high
Security	Control commands to physical equipment and customs data must be protected.	High
Maintainability	Rules, models and integrations will change over time.	High
Interoperability	Must work with existing TOS, crane PLCs, AGV controllers and customs systems.	High

The two most important drivers are real-time responsiveness and continuous availability. Loose coupling between the planning functions also matters, because a vessel delay must update several functions at once.

2. Select a Suitable Software Architecture

2.1 Selected Architecture

The proposed system uses an Event-Driven Microservices Architecture organized as a Hybrid with a Layered structure. Layers (presentation, API gateway, business services, data, integration) keep responsibilities separate. Inside the business layer, independent microservices communicate mainly through events published to a message broker.

2.2 Justification

Reason	Why it suits the seaport platform
Real-time data streams	Crane positions, AGV locations and vessel updates arrive continuously. Event-driven processing reacts as each event arrives instead of waiting for batch runs.
Loose coupling	The slot optimizer, crane scheduler and router do not call each other directly. A delay event published once is consumed by all of them.
Independent scaling	The AGV router can be scaled up in busy periods without scaling the customs service.
Fault isolation	If the analytics service fails, crane scheduling and collision avoidance continue.
Adaptability to delays	A "vessel delayed" event triggers re-planning workflows across berth, crane, yard and truck services.
Integration	Adapters isolate external systems such as the TOS, PLCs and customs systems from the core logic.
Team and technology flexibility	Each service can use the best tool (for example an optimization solver for slot planning).

2.3 Alternatives Considered

Alternative	Reason not chosen alone
Layered (monolith) only	Simple, but hard to scale individual functions and a single failure can stop the whole system.
Client-Server only	Does not handle continuous event streams and many cooperating services well.
MVC only	Suitable for the user interface, but not for distributed real-time orchestration.
Serverless	Cold-start delays and limited control are unsuitable for strict real-time crane safety logic.
Pure SOA with an enterprise bus	Heavier and slower; synchronous calls increase coupling and delay.

3. Software Architecture Diagram

The diagram shows the six layers of the platform, its databases, its communication mechanisms and the external systems it connects to.

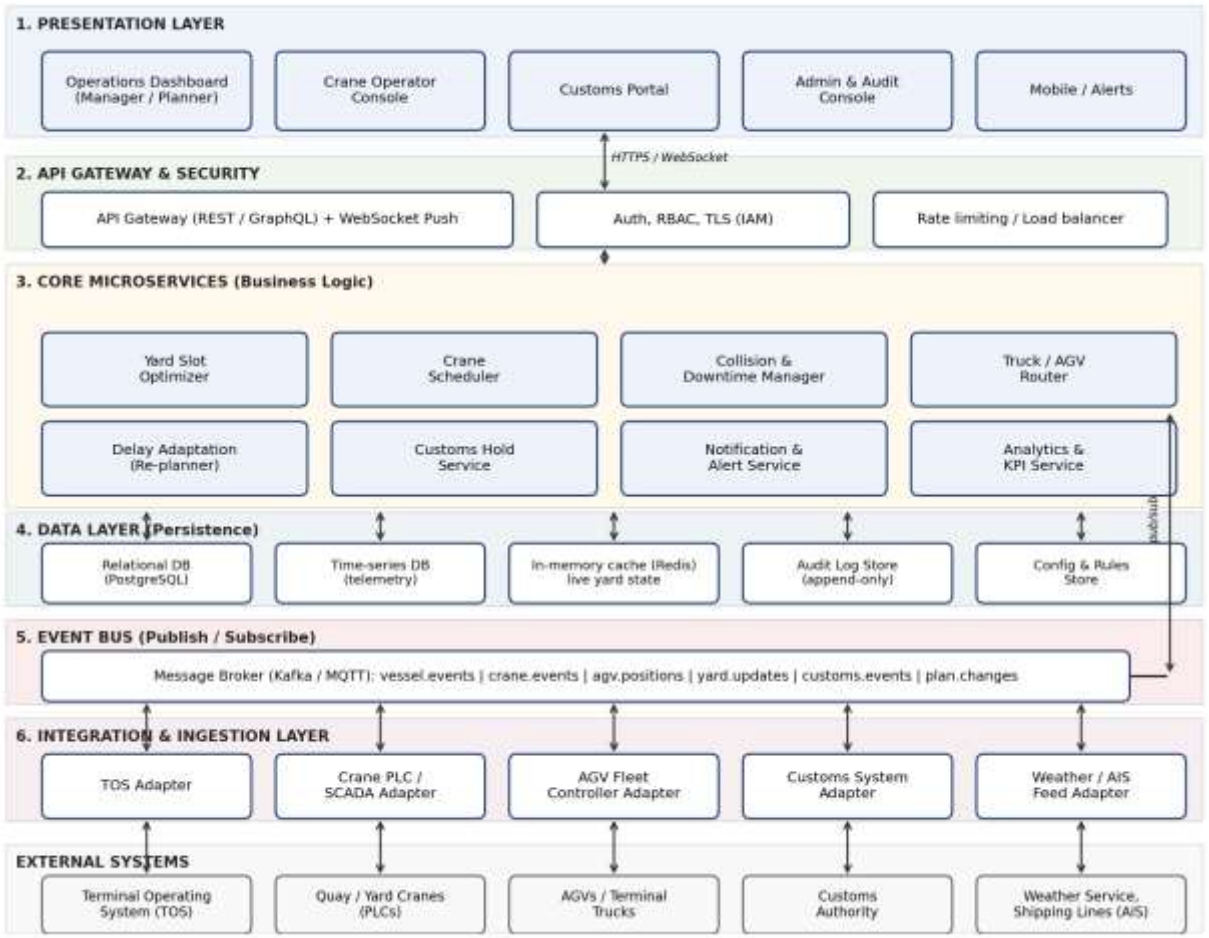


Figure 1: Event-driven microservices architecture of the Seaport Container Yard Coordination Platform.

Communication	**Mechanism**
Users to platform	HTTPS (REST/GraphQL) through the API gateway; WebSocket for live updates
Service to service (asynchronous)	Publish/subscribe events through the message broker (Kafka / MQTT)
Service to service (synchronous, when an answer is needed)	Internal REST/gRPC with timeouts and retries
Platform to external systems	Adapters using TOS APIs, OPC-UA/Modbus for cranes, MQTT/REST for AGVs, EDI/API for customs, feeds for weather/AIS

Services to databases	Database drivers; cache for live yard state

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Purpose and Responsibilities
Presentation Layer	Role-based web dashboards for operations managers, crane operators, planners, customs and administrators; shows live status, alerts and plans.
API Gateway and Security	Single entry point; authentication, authorization (RBAC), TLS, rate limiting, load balancing and WebSocket push.
Integration / Ingestion Adapters	Connect to TOS, crane PLC/SCADA, AGV controllers, customs systems and weather/AIS feeds; convert data into standard events.
Event Bus (Message Broker)	Carries events (vessel, crane, AGV, yard, customs, plan changes) between services; buffers and replays events.
Yard Slot Optimizer	Chooses container slots using departure time, weight, destination and customs status; balances blocks and reduces re-handling.
Crane Scheduler	Calculates crane picking sequences that minimize loading/unloading time and aligns them with AGV pick-up times.
Collision and Downtime Manager	Monitors crane positions and health; detects conflicts, enforces safety clearances, re-sequences or reassigns work when a crane fails.
Truck / AGV Router	Assigns transport jobs and computes dynamic routes; re-routes around congestion and closures.
Delay Adaptation (Re-planner)	Reacts to weather or arrival changes; recalculates berth windows, crane plans, yard reservations and truck jobs.
Customs Hold Service	Records holds, inspections and releases and excludes held containers from pick plans.
Notification and Alert Service	Sends alerts for conflicts, delays, faults and safety events.
Analytics and KPI Service	Calculates turnaround time, moves per hour, utilization and idle time for dashboards and reports.

Data Layer	Relational DB for containers and vessels; time-series DB for telemetry; Redis cache for live yard state (digital twin); append-only audit store; configuration and rules store.

4.2 Data Exchange Between Components

- Adapters publish raw events such as agv.positions or crane.events to the event bus after validation and time-stamping.
- Services subscribe only to the topics they need and publish results such as plan.changes or yard.updates.
- Live yard state is kept in the cache so services read a consistent view quickly; durable data is written to the databases.
- The API gateway reads from services and the cache, and pushes updates to dashboards through WebSocket.
- Approved crane and AGV commands are published to the integration layer, which sends them to the equipment through adapters.

4.3 Overall System Workflow

Step	Activity
1	Shipping line sends the vessel manifest and bay plan; the TOS adapter publishes a vessel.events message.
2	Yard Slot Optimizer allocates slots for containers using departure times and customs status.
3	Crane Scheduler builds the picking sequence; Collision Manager checks it against safety zones.
4	Truck / AGV Router assigns jobs and routes timed to crane output.
5	During operations, crane and AGV telemetry flow through the bus; services update the live yard state and adjust plans.
6	If a weather alert shifts the vessel arrival, Delay Adaptation publishes a revised plan; yard, crane and truck services update automatically.
7	If a crane fails or a collision risk appears, Collision Manager reassigns tasks and the Notification Service alerts operators.
8	Customs places or releases holds; the Customs Service publishes events so held containers are skipped.

9	Dashboards show live status to logistics and customs teams; all decisions are logged to the audit store.

5. Quality Attributes

Quality Attribute	How the Architecture Addresses It
Scalability	Stateless microservices scale horizontally behind a load balancer; the message broker partitions topics by berth or block; databases use replication and partitioning; new berths or cranes are added without redesign.
Security	API gateway with authentication and RBAC; TLS for all traffic; signed and validated commands to equipment; network segmentation between IT and equipment networks; encrypted data at rest; audit logs of all commands and overrides.
Performance	Asynchronous events avoid blocking calls; in-memory cache holds live yard state; time-critical collision logic runs in a dedicated low-latency service; WebSocket pushes updates instead of polling.
Reliability	Service isolation limits failures; the broker stores events so they can be replayed; retries and idempotent handlers prevent duplicate commands; the system falls back to the last valid plan or manual control if inputs are lost.
Maintainability	Small services with clear APIs and versioned configuration; adapters hide changes in external systems; CI/CD and automated testing allow safe updates of one service at a time.
Availability	Redundant service instances and broker clusters across zones; database replication with failover; health checks and automatic restarts; 24x7 monitoring and alerting; rolling upgrades without downtime.

6. Justify Architectural Decisions

6.1 Rationale

- Event-driven design was chosen because yard, crane and vehicle conditions change constantly and many functions must react at once.
- Microservices let cranes, yard planning, routing and customs evolve and scale independently.
- A layered structure keeps user interface, security, business logic, integration and data clearly separated.

- Adapters protect the core logic from vendor-specific equipment interfaces.
- An in-memory digital twin of the yard gives all services a fast, consistent view of current state.

6.2 Trade-offs Considered

Decision	Benefit	Trade-off / Mitigation
Event-driven asynchronous communication	Loose coupling, fast reaction, easy extension	Harder debugging and eventual consistency; mitigated with event tracing, ordering keys and idempotent handlers.
Microservices instead of a monolith	Independent scaling and fault isolation	More operational complexity; mitigated with containers, orchestration (Kubernetes) and monitoring.
Message broker as central component	Buffering, replay, decoupling	Potential single point of failure; mitigated with a clustered broker.
Separate real-time collision service	Fast, safety-critical response	Extra component to maintain; mitigated by keeping it small and well tested.
Cache for live state	Very fast reads	Risk of stale data; mitigated with time-stamps and write-through updates to the database.

6.3 Support for Future Enhancements, Integration and Long-Term Maintainability

- New capabilities (predictive maintenance, machine-learning arrival forecasting, rail/truck gate scheduling, carbon-emission tracking) can be added as new services subscribing to existing events.
- New equipment or systems are integrated by writing a new adapter without changing core services.
- Standard APIs and event schemas allow integration with other ports, shipping lines and national customs platforms.
- A digital twin and simulation service can use recorded events to test plans before applying them.
- Versioned services and configuration allow gradual upgrades and rollback.
- Clear separation of responsibilities lets teams work in parallel and keeps the system maintainable over many years.

Conclusion

An Event-Driven Microservices Architecture with a layered structure best meets the needs of the Seaport Container Yard Coordination Platform. It supports real-time coordination of containers, cranes and trucks, adapts quickly to vessel delays, meets requirements for performance, reliability, availability, scalability, security and maintainability, and allows the port to increase capacity without expanding its physical footprint.